

Dokumentacja Techniczna Projektu: SwimSync

1. Wprowadzenie

SwimSync to system backendowy napisany w języku Python, służący do zarządzania infrastrukturą szkoły pływania. System obsługuje profile użytkowników, rezerwacje zajęć, weryfikację dostępności torów basenowych oraz planowanie harmonogramu instruktorów.

Aplikacja nie wykorzystuje zewnętrznych silników bazodanowych (np. SQL) – zamiast tego implementuje własną warstwę persystencji opartą na plikach **JSON**, co czyni ją niezwykle lekką i łatwą do uruchomienia w każdym środowisku. Architektura opiera się na separacji warstw (Encje - > Baza Danych -> Kontrolery -> Interfejsy).

2. Struktura Projektu

Projekt składa się z następujących głównych plików:

- **entities.py** - Warstwa Domenowa (logika obiektów).
- **database.py** - Warstwa Persystencji (operacje I/O na plikach JSON).
- **controllers.py** - Warstwa Logiki Biznesowej (walidacje, przetwarzanie żądań).
- **boundaries.py** - Warstwa Prezentacji (symulacja interfejsu użytkownika / API).
- **test_swimsync.py** - Zestaw testów jednostkowych i integracyjnych (pytest).

3. Opis Modułów i Kodu (API Reference)

3.1. entities.py (Encje)

Moduł definiuje struktury danych. Każda klasa posiada metody `to_dict()` oraz `from_dict()` umożliwiające łatwą serializację do formatu JSON.

- **ProfilUzytkownika**: Przechowuje dane użytkowników. Metoda `generate_temp_password()` generuje 10-znakowy losowy ciąg alfanumeryczny.
- **TorBasenowy**: Określa zasoby fizyczne basenu. Przechowuje bezwzględny limit pojemności (`max_pojemnosc`).
- **GrupaPlywacka**: Definiuje parametry grupy. Kluczowa metoda `sprawdzWolneMiejsca()` dynamicznie kalkuluje bezpieczny limit, porównując pojemność toru z maksymalną wielkością grupy.
- **Rezerwacja**: Przechowuje relację Kursant-Zajęcia. Wdrożono tu mechanizm **Soft Delete** za pomocą metody `zmienStatus()` (zamiast fizycznie usuwać rezerwację, system oznacza ją jako "anulowana").
- **Obecnosc**: Reprezentuje status obecności kursanta na konkretnych zajęciach.

- **Harmonogram:** Moduł rejestrujący czasowe blokady instruktorów.
- **Raport:** Klasa agregująca dane w pamięci (np. zliczająca frekwencję lub przepracowane godziny instruktorów) i symulująca generowanie plików (np. PDF).

3.2. database.py (Baza Danych)

Moduł odpowiedzialny za operacje na systemie plików.

- **Zarządzanie Plikami:** Klasa BazaDanych automatycznie tworzy katalog data/ i inicjalizuje w nim pliki JSON (m.in. uzytkownicy.json, zajecia.json, rezerwacje.json), jeśli te nie istnieją.
- **Operacje I/O:** Metody `_read_file()` i `_write_file()` odpowiadają za bezpieczny odczyt i zapis danych z odpowiednim kodowaniem UTF-8.
- **CRUD:** Implementacja metod takich jak `save()`, `insert()` (automatycznie generująca ID w zakresie 1000-9999), oraz specjalistycznych zapytań, np. `pobierzNakladajaceSieZajecia()`, która konwertuje daty z formatu ISO i wyszukuje konflikty w czasie.
- **Bulk Insert:** Metoda `bulk_insert()` pozwala na zrzut wielu obiektów (np. list obecności) do pliku w ramach jednego zapisu, co optymalizuje wydajność systemu.

3.3. controllers.py (Logika Biznesowa)

Serce aplikacji decydujące o przepływie danych. Zwraca wyjątki (`ValueError`, `UserWarning`) w przypadku złamania reguł biznesowych.

- **ZarządzanieController:** Waliduje m.in. zmianę wielkości torów. Metoda `_sprawdzWielkoscGrup()` zapobiega zmniejszeniu toru poniżej liczby już zapisanych na nim osób.
- **RezerwacjeController:**
 - `procesujZapis()` weryfikuje wolne miejsca w grupie.
 - `procesujAnulowanie()` wylicza różnicę czasu. Zezwala na bezkosztowe anulowanie zajęć tylko w przypadku wyprzedzenia wynoszącego co najmniej **24 godziny**.
- **HarmonogramController:** `sprawdzIZapiszHarmonogram()` blokuje administratora przed przypisaniem dwóch grup do tego samego toru lub instruktora o tej samej godzinie.
- **ObecnoscController:** Wykorzystuje matematyczne **operacje na zbiorach (set)** do błyskawicznego wyliczenia listy osób nieobecnych na podstawie różnicy zapisanych kursantów i dostarczonej listy osób obecnych.
- **PowiadomieniaController:** Posiada funkcje rozsyłania alertów o zmianach w grafiku oraz przypomnień (np. `cronJob_sprawdzHarmonogram()`).

3.4. boundaries.py (Warstwa Interfejsu)

Stanowi punkt wejściowy do aplikacji. Kontroluje komunikację z użytkownikiem (w tym środowisku – poprzez rzut na konsolę za pomocą funkcji `print()`).

- Przechwytuje wyjątki (np. `try ... except ValueError as e:`) wyrzucane przez kontrolery.
- Symuluje wyświetlanie popupów, błędów formularzy oraz okien pobierania przeglądarki. Zawiera klasy `InterfejsWebowy`, `PortalZapisow`, `SystemAplikacja` oraz `SystemCron`.

4. Środowisko Testowe i Testy Automatyczne

System został objęty szczegółowymi testami za pomocą frameworka **pytest**. Plik `test_swimsync.py` zawiera asercje dla wszystkich warstw architektury.

4.1. Architektura Testów i Fixtures

Aby chronić prawdziwe dane produkcyjne (katalog `data/`), środowisko testowe wykorzystuje mechanizm tymczasowych, odizolowanych katalogów.

- **temp_db(tmp_path):** Wstrzykuje wirtualny katalog operacyjny. Dzięki temu testy operują na plikach tworzonych i niszczonych w locie.
- **controllers(temp_db):** Automatycznie instancjonuje wszystkie kontrolery, wiążąc je z tymczasową bazą danych, przygotowując pełne środowisko dla każdego testu.

4.2. Przeprowadzone Scenariusze Testowe

- **Testy Logiki Domenowej (TestEntities):** Weryfikacja kalkulacji miejsc w grupach (`sprawdzWolneMiejsca`), generowania haseł, zmian statusów oraz agregacji wyników do raportów.
- **Testy Bazy Danych (TestDatabase):** Sprawdzanie fizycznego tworzenia plików, weryfikacja poprawności serializacji i operacji `bulk_insert`.
- **Testy Logiki Biznesowej (TestZarzadzanieController, TestRezerwacjeController, TestHarmonogramController):**
 - Pomyślne rejestrowanie działań (Happy Path).
 - Wyłapywanie `ValueError` przy próbach: zapisu do pełnej grupy, tworzenia konfliktów na torach w harmonogramie, spóźnionego anulowania zajęć (poniżej 24h).
 - Wyłapywanie alertów (`UserWarning`) przy nakładaniu się urlopu instruktora z jego przypisanymi zajęciami.
- **Testy Prezentacji (TestBoundaries):** Przy użyciu wbudowanego w `pytest` mechanizmu `capsys` testy weryfikują, czy odpowiednie komunikaty wędrują do standardowego wyjścia konsoli (`stdout`) jako odpowiedź dla użytkownika.